

Feature Envy GeneralSearch

GeneralSearch

1. Foi enviado para a IA o código da classe GeneralSearch com o método handleSearch com o seguinte comando:

```
results.addAll(StudyTaskManager.getStudyTaskManager().searchInRegistries(text));
this.searchLog.addSearchHistory(text);
this.searchLog.setNumUsages(this.searchLog.getNumUsages() + 1);
results.add("\nLogged in: " + this.searchLog.getLogName());
return results;
}
```

refatore o code smell Feature Envy utilizando a técnica de refatoração move method para a classe que enviarei a seguir.

O code smell "Feature Envy" ocorre quando um método usa mais os atributos e métodos de outra classe do que da própria classe onde está localizado. Para resolver isso, podemos mover o método `handleSearch` para a classe mais apropriada que centraliza as buscas, dependendo do contexto da aplicação.

Antes de fazer a refatoração, precisamos analisar as classes mencionadas no método `handleSearch`. Compartilhe, por favor, a definição de pelo menos uma das classes referenciadas (`CardManager`, `HabitTracker`, `TodoTracker`, `StudyMaterial`, ou `StudyTaskManager`) ou o contexto de qual classe deve ser responsável pela lógica de busca consolidada.

Com base nisso, faremos a refatoração utilizando a técnica de move method.

2. Após isso, enviei o código do SearchLog e solicitei o Move to:

```
public void setNumUsages(int numUsages) {
    this.numUsages = numUsages;
}
```

Move to

Para eliminar o code smell **Feature Envy** presente no método `handleSearch` da classe `GeneralSearch`, vamos movê-lo para a classe `SearchLog`. O método será transformado em um mecanismo centralizado para registrar e gerenciar buscas enquanto interage com as classes relacionadas. Essa refatoração centraliza a responsabilidade da busca em `SearchLog`, pois esta já gerencia o histórico e estatísticas de buscas.

Abaixo está o código refatorado:

Refatoração em `SearchLog`

```
java
package org.example.studysearch;

import java.util.*;

public class SearchLog {
```

3. A IA me retornou as 2 classes de uma vez, já refatoradas:

```
//REFATORAÇÃO SEARCHLOG

package org.example.studysearch;

import java.util.*;

public class SearchLog {
    private final List<String> searchHistory;
    private final Map<String, Integer> searchCount;
    private boolean isLocked;
    private int numUsages;
```

```

private String logName;

public SearchLog(String logName) {
    this.searchHistory = new ArrayList<>();
    this.searchCount = new HashMap<>();
    this.logName = logName;
    this.numUsages = 0;
    this.isLocked = false;
}

// Método movido e refatorado de GeneralSearch
public List<String> handleSearch(String text) {
    if (isLocked) {
        throw new IllegalStateException("SearchLog is locked. Cannot perform a search.");
    }
    List<String> results = new ArrayList<>();
    results.addAll(CardManager.getCardManager().searchInCards(text));
    results.addAll(HabitTracker.getHabitTracker().searchInHabits(text));
    results.addAll(TodoTracker.getInstance().searchInTodos(text));
    results.addAll(StudyMaterial.getStudyMaterial().searchInMaterials(text));
    results.addAll(StudyTaskManager.getStudyTaskManager().searchInRegistries(text));

    logSearch(text); // Usa o método encapsulado para registrar buscas
    results.add("\nLogged in: " + this.logName);
    return results;
}

// Encapsulate adding a search term and updating its count
public void logSearch(String term) {
    if (isLocked) {
        throw new IllegalStateException("Cannot log searches; the log is locked.");
    }

    searchHistory.add(term);
    searchCount.put(term, searchCount.getOrDefault(term, 0) + 1);
    numUsages++;
}

// Outros métodos permanecem inalterados...
}

```

//REFATORAÇÃO GLOBALSEARCH

```

package org.example.studysearch;

import org.example.studycards.CardManager;
import org.example.studyplanner.HabitTracker;
import org.example.studyplanner.TodoTracker;
import org.example.studyregistry.StudyMaterial;
import org.example.studyregistry.StudyTaskManager;

import java.util.ArrayList;
import java.util.List;

public class GeneralSearch implements Search<String> {
    private final SearchLog searchLog = new SearchLog("General Search");

    public GeneralSearch() {
    }

    @Override
    public List<String> search(String text) {
        return searchLog.handleSearch(text); // Delega a lógica para SearchLog
    }
}

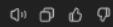
```

```
public SearchLog getSearchLog() {  
    return searchLog;  
}  
}
```

Benefícios da Refatoração

1. **Centralização de responsabilidade:** `SearchLog` agora gerencia tanto o histórico quanto a execução da busca.
2. **Maior coesão:** O método `handleSearch` está diretamente relacionado ao propósito de `SearchLog`.
3. **Melhor testabilidade:** O `SearchLog` pode ser testado de forma independente.
4. **Redução de acoplamento:** `GeneralSearch` é mais simples e atua apenas como um ponto de entrada para buscas.

Se precisar de mais ajustes ou contexto adicional, me avise!



3. Código livre do code smell e funcionando

Feature Envy **StudyCardsController**

StudyCardsController

1. Foi enviado para a IA o código da classe StudyCardsController com o método getRandomCardFromAllBoxes e da classe e LeitnerSystem com o seguinte comando:

```
terBox.removeCard(cardId);
boxes.get(Math.max(boxId - 1, 0)).addCard(cardId);
}
}
```

refatore o code smell Feature Envy utilizando a técnica de refatoração move method.

Para refatorar o code smell **Feature Envy** no código fornecido, utilizando a técnica de refatoração **Move Method**, é necessário identificar métodos que estão utilizando frequentemente atributos ou métodos de outra classe, sugerindo que eles talvez pertençam àquela outra classe.

Problema Identificado

Os métodos `upgradeCard` e `downgradeCard` na classe `LeitnerSystem` têm lógica que depende principalmente de operações sobre objetos da classe `Box`. Essas operações seriam mais apropriadas na classe `Box`, reduzindo o acoplamento entre as duas classes.

Aqui está a refatoração:

Alterações em `LeitnerSystem`

1. Movemos a lógica de validação e operações específicas para `Box`.
2. Alteramos `LeitnerSystem` para delegar essas operações.

Resultados:

```
public void upgradeCard(Integer cardId, Integer boxId) throws Exception {
    boxIdValidation(boxId);
    Box currentBox = boxes.get(boxId);
    Box nextBox = boxes.get(Math.min(boxId + 1, 4));
    currentBox.moveCardTo(cardId, nextBox);
}

public void downgradeCard(Integer cardId, Integer boxId) throws Exception {
    boxIdValidation(boxId);
    Box currentBox = boxes.get(boxId);
    Box previousBox = boxes.get(Math.max(boxId - 1, 0));
    currentBox.moveCardTo(cardId, previousBox);
}
```

```
public class Box {
    private List<Integer> cards;

    public Box() {
        this.cards = new ArrayList<>();
    }

    public void addCard(Integer cardId) {
        if (!cards.contains(cardId)) {
            cards.add(cardId);
        }
    }

    public void removeCard(Integer cardId) {
        cards.remove(cardId);
    }
}
```

```

public boolean hasCard(Integer cardId) {
    return cards.contains(cardId);
}

public Integer getRandomCard() {
    if (cards.isEmpty()) {
        return null;
    }
    return cards.get((int) (Math.random() * cards.size()));
}

public void addCards(List<Integer> otherCards) {
    for (Integer cardId : otherCards) {
        if (!cards.contains(cardId)) {
            cards.add(cardId);
        }
    }
}

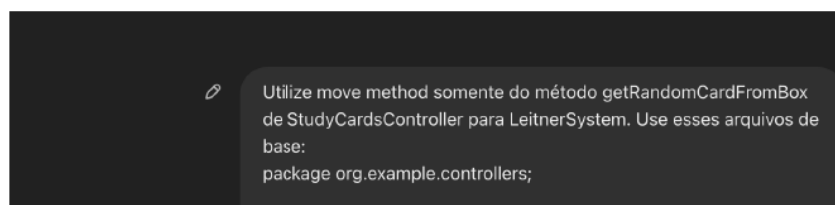
public void moveCardTo(Integer cardId, Box targetBox) throws Exception {
    if (!this.hasCard(cardId)) {
        throw new Exception("Card not found in the current box.");
    }
    this.removeCard(cardId);
    targetBox.addCard(cardId);
}

@Override
public String toString() {
    return cards.toString();
}

public List<Integer> getCards() {
    return new ArrayList<>(cards);
}
}

```

2. A IA fez alterações indevidas, em métodos diferentes do problemático, então tivemos que ser mais específicos e tornamos a enviar o código completo dos dois métodos:



Resultados:



Para refatorar o método `getRandomCardFromBox` da classe `StudyCardsController` para a classe `LeitnerSystem`, utilizando a técnica `Move Method`, faremos as seguintes mudanças:

Refatoração

1. Modificação na Classe `LeitnerSystem`

O método `getRandomCardFromBox` será movido para a classe `LeitnerSystem`. Ele será renomeado para algo mais coeso, como `getRandomCardFromAllBoxes`.

java

Copy code

```
public class LeitnerSystem extends StudyMethod {
    // Outros métodos e atributos permanecem os mesmos

    public String getRandomCardFromAllBoxes() {
        StringBuilder response = new StringBuilder();
        response.append(getMethodName()).append("\n");
        List<Box> boxes = getBoxes();
        response.append(getRandomCard(boxes));
        return response.toString();
    }
}
```

2. Modificação na Classe `StudyCardsController`

Na classe `StudyCardsController`, o método `getRandomCardFromBox` agora apenas delegará a lógica para a classe `LeitnerSystem`.

java

Copy code

```
public class StudyCardsController {
    // Outros métodos e atributos permanecem os mesmos

    public String getRandomCardFromBox() {
        return leitnerSystem.getRandomCardFromAllBoxes();
    }
}
```

Benefícios da Refatoração

1. Redução do Feature Envy:

- O método movido interagia intensivamente com os métodos e atributos da classe `LeitnerSystem`, o que indicava que ele estava no lugar errado.
- Agora, ele pertence à classe onde essa lógica é mais coesa.

2. Aumento da Coesão:

- Toda lógica relacionada à manipulação e consulta das caixas está encapsulada na classe `LeitnerSystem`.

3. Código mais limpo e fácil de manter:

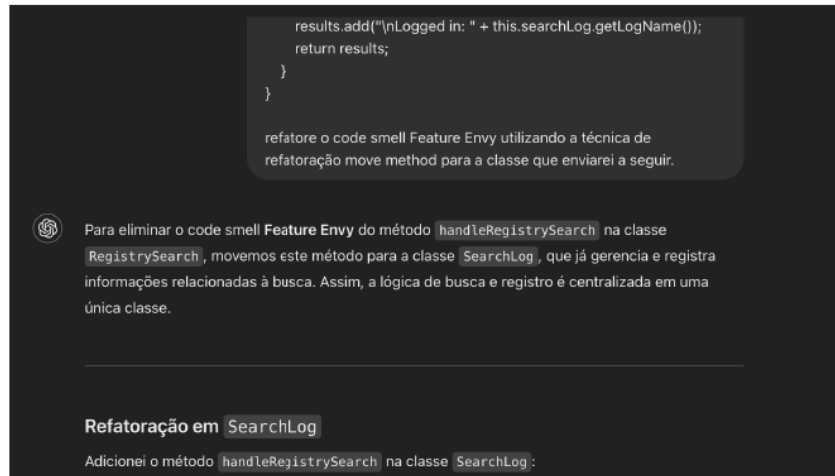


3. Código livre do code smell e funcionando

Feature Envy RegistrySearch

RegistrySearch

1. Foi enviado para a IA o código da classe RegistrySearch com o método handleRegistrySearch com o seguinte comando:



```
package org.example.studysearch;

import org.example.studycards.CardManager;
import org.example.studyplanner.HabitTracker;
import org.example.studyplanner.TODOTracker;
import org.example.studyregistry.StudyTaskManager;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class SearchLog {
    private final List<String> searchHistory;
    private final Map<String, Integer> searchCount;
    private boolean isLocked;
    private int numUsages;
    private String logName;

    public SearchLog(String logName) {
        this.searchHistory = new ArrayList<>();
        this.searchCount = new HashMap<>();
        this.logName = logName;
        this.numUsages = 0;
        this.isLocked = false;
    }

    public List<String> handleRegistrySearch(String text) {
        if (isLocked) {
            throw new IllegalStateException("SearchLog is locked. Cannot perform a search.");
        }
        List<String> results = new ArrayList<>();
        results.addAll(CardManager.getCardManager().searchInCards(text));
        results.addAll(HabitTracker.getHabitTracker().searchInHabits(text));
        results.addAll(TODOTracker.getInstance().searchInTodos(text));
        results.addAll(StudyTaskManager.getStudyTaskManager().searchInRegistries(text));

        logSearch(text);
    }
}
```

```

        results.add("\nLogged in: " + this.logName);
        return results;
    }

    public void logSearch(String term) {
        if (isLocked) {
            throw new IllegalStateException("Cannot log searches; the log is locked.");
        }

        searchHistory.add(term);
        searchCount.put(term, searchCount.getOrDefault(term, 0) + 1);
        numUsages++;
    }

    public List<String> getSearchHistory() {
        return Collections.unmodifiableList(searchHistory);
    }

    public int getSearchCount(String term) {
        return searchCount.getOrDefault(term, 0);
    }

    public List<String> getTopSearches() {
        int maxCount = searchCount.values().stream().max(Integer::compareTo).orElse(0);
        List<String> topSearches = new ArrayList<>();
        for (Map.Entry<String, Integer> entry : searchCount.entrySet()) {
            if (entry.getValue() == maxCount) {
                topSearches.add(entry.getKey());
            }
        }
        return topSearches;
    }

    public void lock() {
        isLocked = true;
    }

    public void unlock() {
        isLocked = false;
    }

    public int getNumUsages() {
        return numUsages;
    }

    public void setNumUsages(int numUsages) {
        this.numUsages = numUsages;
    }

    public String getLogName() {
        return logName;
    }

    public void setLogName(String logName) {
        if (logName == null || logName.isBlank()) {
            throw new IllegalArgumentException("Log name cannot be null or blank.");
        }
        this.logName = logName;
    }
}

```

```

package org.example.studysearch;

import java.util.List;

```



```
public class RegistrySearch implements Search<String> {

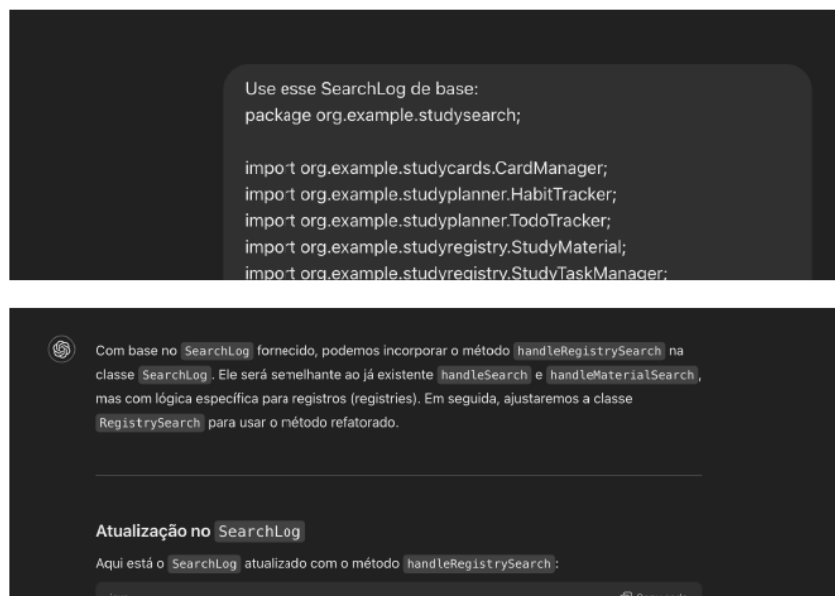
    private final SearchLog searchLog = new SearchLog("Registry Search");

    public RegistrySearch() {}

    @Override
    public List<String> search(String text) {
        return searchLog.handleRegistrySearch(text); // Delega a busca ao SearchLog
    }

    public SearchLog getSearchLog() {
        return searchLog;
    }
}
```

2. A IA gerou corretamente a classe RegistrySearch mas usou a classe SearchLog errada de base. Foi aproveitada a classe ResgistrySearch e solicitado que gerasse novamente a SearchLog com a base correta:



3. A IA me retornou as 2 classes de uma vez, já refactoradas:

```
package org.example.studysearch;

import org.example.studycards.CardManager;
import org.example.studyplanner.HabitTracker;
import org.example.studyplanner.TODOTracker;
import org.example.studyregistry.StudyMaterial;
import org.example.studyregistry.StudyTaskManager;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class SearchLog {
    private final List<String> searchHistory;
    private final Map<String, Integer> searchCount;
    private boolean isLocked;
    private int numUsages;
    private String logName;
```

```

public SearchLog(String logName) {
    this.searchHistory = new ArrayList<>();
    this.searchCount = new HashMap<>();
    this.logName = logName;
    this.numUsages = 0;
    this.isLocked = false;
}

public List<String> handleSearch(String text) {
    if (isLocked) {
        throw new IllegalStateException("SearchLog is locked. Cannot perform a search.");
    }
    List<String> results = new ArrayList<>();
    results.addAll(CardManager.getCardManager().searchInCards(text));
    results.addAll(HabitTracker.getHabitTracker().searchInHabits(text));
    results.addAll(TodoTracker.getInstance().searchInTodos(text));
    results.addAll(StudyMaterial.getStudyMaterial().searchInMaterials(text));
    results.addAll(StudyTaskManager.getStudyTaskManager().searchInRegistries(text));

    logSearch(text);
    results.add("\nLogged in: " + this.logName);
    return results;
}

public List<String> handleMaterialSearch(String text) {
    if (isLocked) {
        throw new IllegalStateException("SearchLog is locked. Cannot perform a search.");
    }
    List<String> results = new ArrayList<>();
    results.addAll(StudyMaterial.getStudyMaterial().searchInMaterials(text));

    logSearch(text);
    results.add("\nLogged in: " + this.logName);
    return results;
}

public List<String> handleRegistrySearch(String text) {
    if (isLocked) {
        throw new IllegalStateException("SearchLog is locked. Cannot perform a search.");
    }
    List<String> results = new ArrayList<>();
    results.addAll(CardManager.getCardManager().searchInCards(text));
    results.addAll(HabitTracker.getHabitTracker().searchInHabits(text));
    results.addAll(TodoTracker.getInstance().searchInTodos(text));
    results.addAll(StudyTaskManager.getStudyTaskManager().searchInRegistries(text));

    logSearch(text);
    results.add("\nLogged in: " + this.logName);
    return results;
}

public void logSearch(String term) {
    if (isLocked) {
        throw new IllegalStateException("Cannot log searches; the log is locked.");
    }

    searchHistory.add(term);
    searchCount.put(term, searchCount.getOrDefault(term, 0) + 1);
    numUsages++;
}

public List<String> getSearchHistory() {
    return Collections.unmodifiableList(searchHistory);
}

```

```

    }

    public int getSearchCount(String term) {
        return searchCount.getDefault(term, 0);
    }

    public List<String> getTopSearches() {
        int maxCount = searchCount.values().stream().max(Integer::compareTo).orElse(0);
        List<String> topSearches = new ArrayList<>();
        for (Map.Entry<String, Integer> entry : searchCount.entrySet()) {
            if (entry.getValue() == maxCount) {
                topSearches.add(entry.getKey());
            }
        }
        return topSearches;
    }

    public void lock() {
        isLocked = true;
    }

    public void unlock() {
        isLocked = false;
    }

    public int getNumUsages() {
        return numUsages;
    }

    public String getLogName() {
        return logName;
    }

    public void setLogName(String logName) {
        if (logName == null || logName.isBlank()) {
            throw new IllegalArgumentException("Log name cannot be null or blank.");
        }
        this.logName = logName;
    }

    public void addSearchHistory(String query) {
        searchHistory.add(query);
    }

    public void setNumUsages(int numUsages) {
        this.numUsages = numUsages;
    }
}

```

```

package org.example.studysearch;

import java.util.List;

public class RegistrySearch implements Search<String> {
    private final SearchLog searchLog = new SearchLog("Registry Search");

    public RegistrySearch() {}

    @Override
    public List<String> search(String text) {
        return searchLog.handleRegistrySearch(text); // Delega a busca ao SearchLog
    }

    public SearchLog getSearchLog() {

```

```
        return searchLog;
    }
}
```

Benefícios da Refatoração

1. Centralização de Responsabilidade:

- A lógica de busca específica para registros agora reside no `SearchLog`, que já é responsável por gerenciar e registrar buscas.

2. Reutilização:

- O método `handleRegistrySearch` pode ser reutilizado por outras classes, se necessário.

3. Redução de Duplicação:

- Removemos a duplicação de código em `RegistrySearch`, transferindo a lógica para um método reutilizável.

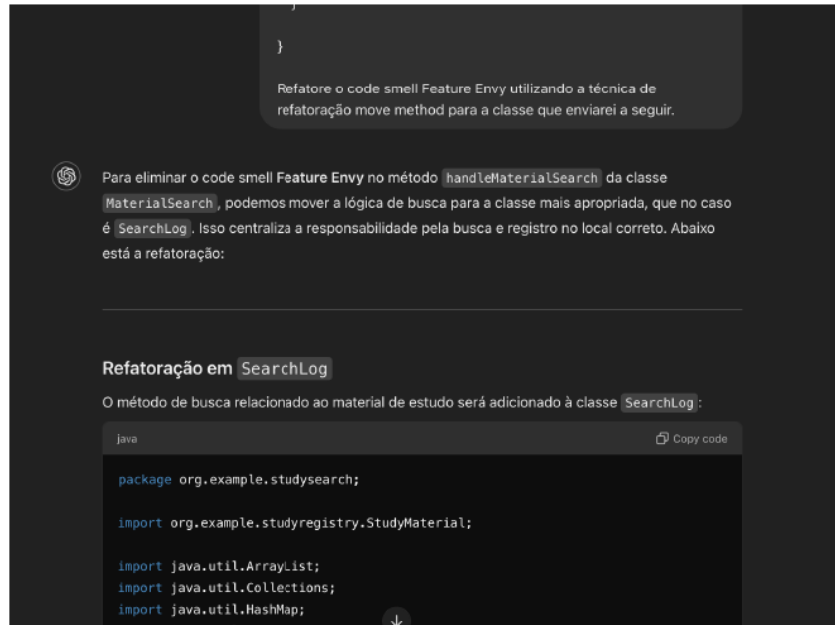
Se precisar de mais ajustes ou suporte, estou à disposição!

3. Código livre do code smell e funcionando

Feature Envy MaterialSearch

MaterialSearch

1. Foi enviado para a IA o código da classe MaterialSearch com o método handleMaterialSearch com o seguinte comando:



```
package org.example.studysearch;

import org.example.studyregistry.StudyMaterial;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class SearchLog {
    private final List<String> searchHistory;
    private final Map<String, Integer> searchCount;
    private boolean isLocked;
    private int numUsages;
    private String logName;

    public SearchLog(String logName) {
        this.searchHistory = new ArrayList<>();
        this.searchCount = new HashMap<>();
        this.logName = logName;
        this.numUsages = 0;
        this.isLocked = false;
    }

    public List<String> searchInMaterials(String text) {
        if (isLocked) {
            throw new IllegalStateException("SearchLog is locked. Cannot perform a search.");
        }

        List<String> results = new ArrayList<>();
        results.addAll(StudyMaterial.getStudyMaterial().searchInMaterials(text));
        logSearch(text); // Log the search term and update usage stats
        results.add("\nLogged in: " + this.logName);
    }
}
```

```

        return results;
    }

    public void logSearch(String term) {
        if (isLocked) {
            throw new IllegalStateException("Cannot log searches; the log is locked.");
        }

        searchHistory.add(term);
        searchCount.put(term, searchCount.getOrDefault(term, 0) + 1);
        numUsages++;
    }

    public List<String> getSearchHistory() {
        return Collections.unmodifiableList(searchHistory);
    }

    public int getSearchCount(String term) {
        return searchCount.getOrDefault(term, 0);
    }

    public int getNumUsages() {
        return numUsages;
    }

    public void setNumUsages(int numUsages) {
        this.numUsages = numUsages;
    }

    public String getLogName() {
        return logName;
    }

    public void setLogName(String logName) {
        this.logName = logName;
    }
}

```

```

package org.example.studysearch;

import java.util.List;

public class MaterialSearch implements Search<String> {

    private final SearchLog searchLog = new SearchLog("Material Search");

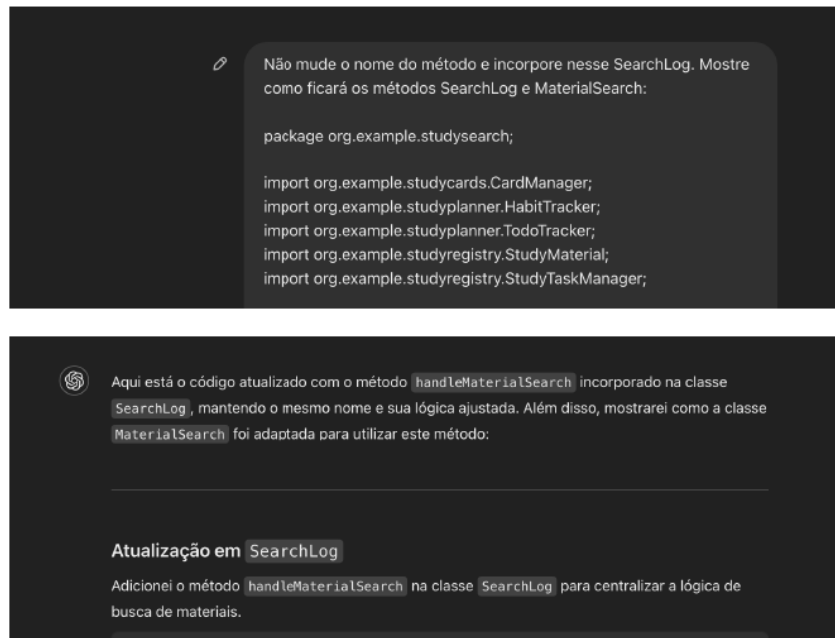
    public MaterialSearch() {}

    @Override
    public List<String> search(String text) {
        return searchLog.searchInMaterials(text); // Delega a busca ao SearchLog
    }

    public SearchLog getSearchLog() {
        return searchLog;
    }
}

```

2. A IA modificou o nome do método e gerou a SearchLog errada. Foi então solicitado que não mudasse o nome do método e entregue a SearchLog certa de base.



3. A IA me retornou as 2 classes de uma vez, já refatoradas:

```
package org.example.studysearch;

import org.example.studycards.CardManager;
import org.example.studyplanner.HabitTracker;
import org.example.studyplanner.TODOTracker;
import org.example.studyregistry.StudyMaterial;
import org.example.studyregistry.StudyTaskManager;

import java.util.*;

public class SearchLog {
    private final List<String> searchHistory;
    private final Map<String, Integer> searchCount;
    private boolean isLocked;
    private int numUsages;
    private String logName;

    public SearchLog(String logName) {
        this.searchHistory = new ArrayList<>();
        this.searchCount = new HashMap<>();
        this.logName = logName;
        this.numUsages = 0;
        this.isLocked = false;
    }

    public List<String> handleSearch(String text) {
        if (isLocked) {
            throw new IllegalStateException("SearchLog is locked. Cannot perform a search.");
        }
        List<String> results = new ArrayList<>();
        results.addAll(CardManager.getCardManager().searchInCards(text));
        results.addAll(HabitTracker.getHabitTracker().searchInHabits(text));
        results.addAll(TODOTracker.getInstance().searchInTodos(text));
        results.addAll(StudyMaterial.getStudyMaterial().searchInMaterials(text));
        results.addAll(StudyTaskManager.getStudyTaskManager().searchInRegistries(text));

        logSearch(text);
        results.add("\nLogged in: " + this.logName);
        return results;
    }
}
```

```

    }

    public List<String> handleMaterialSearch(String text) {
        if (isLocked) {
            throw new IllegalStateException("SearchLog is locked. Cannot perform a search.");
        }
        List<String> results = new ArrayList<>();
        results.addAll(StudyMaterial.getStudyMaterial().searchInMaterials(text));

        logSearch(text);
        results.add("\nLogged in: " + this.logName);
        return results;
    }

    public void logSearch(String term) {
        if (isLocked) {
            throw new IllegalStateException("Cannot log searches; the log is locked.");
        }

        searchHistory.add(term);
        searchCount.put(term, searchCount.getOrDefault(term, 0) + 1);
        numUsages++;
    }

    public List<String> getSearchHistory() {
        return Collections.unmodifiableList(searchHistory);
    }

    public int getSearchCount(String term) {
        return searchCount.getOrDefault(term, 0);
    }

    public List<String> getTopSearches() {
        int maxCount = searchCount.values().stream().max(Integer::compareTo).orElse(0);
        List<String> topSearches = new ArrayList<>();
        for (Map.Entry<String, Integer> entry : searchCount.entrySet()) {
            if (entry.getValue() == maxCount) {
                topSearches.add(entry.getKey());
            }
        }
        return topSearches;
    }

    public void lock() {
        isLocked = true;
    }

    public void unlock() {
        isLocked = false;
    }

    public int getNumUsages() {
        return numUsages;
    }

    public String getLogName() {
        return logName;
    }

    public void setLogName(String logName) {
        if (logName == null || logName.isBlank()) {
            throw new IllegalArgumentException("Log name cannot be null or blank.");
        }
        this.logName = logName;
    }

```



```

    }

    public void addSearchHistory(String query) {
        searchHistory.add(query);
    }

    public void setNumUsages(int numUsages) {
        this.numUsages = numUsages;
    }
}

```

```

package org.example.studysearch;

import java.util.List;

public class MaterialSearch implements Search<String> {

    private final SearchLog searchLog = new SearchLog("Material Search");

    public MaterialSearch() {}

    @Override
    public List<String> search(String text) {
        return searchLog.handleMaterialSearch(text); // Delega a busca ao SearchLog
    }

    public SearchLog getSearchLog() {
        return searchLog;
    }
}

```

Benefícios da Refatoração

1. Centralização de Lógica:

- O método `handleMaterialSearch` foi movido para `SearchLog`, onde se concentra a lógica de registro e busca.

2. Maior Coesão:

- A classe `SearchLog` encapsula comportamentos relacionados a busca e registro.
- A classe `MaterialSearch` agora é mais leve e simplesmente delega.

3. Reutilização:

- Qualquer classe pode agora reutilizar o método `handleMaterialSearch` diretamente do `SearchLog`, eliminando duplicação.

Se precisar de mais ajuda ou modificações, estou à disposição!

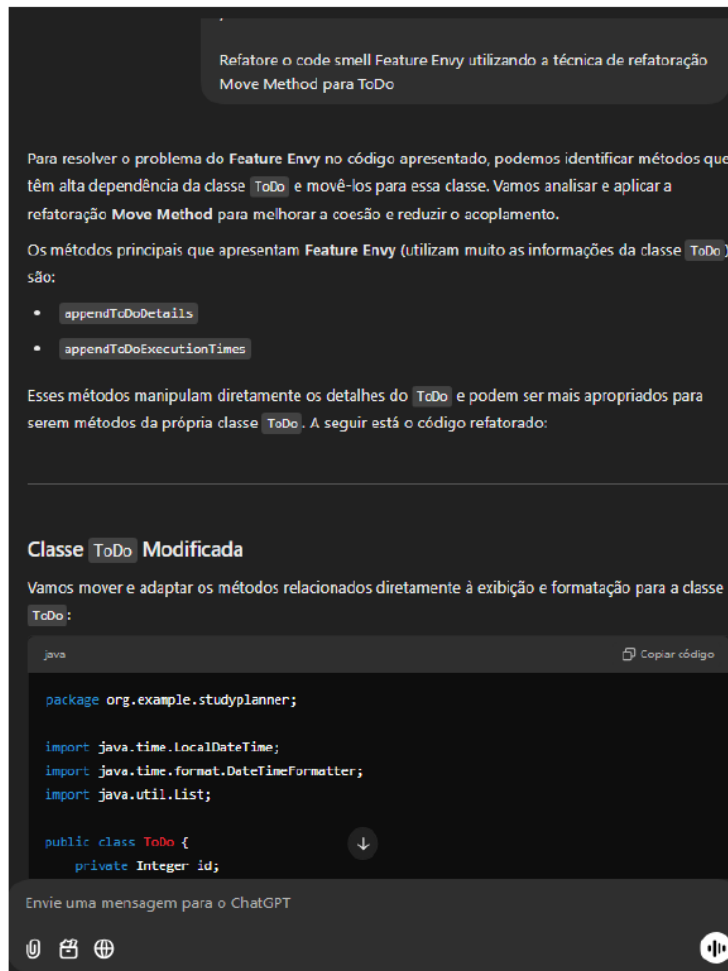


3. Código livre do code smell e funcionando

Feature Envy **ToDoTracker** (efeito colateral)

ToDoTracker

1. Foi enviado para a IA o código ToDoTracker com o seguinte comando:



```
//REFATORAÇÃO DA IA ToDo

package org.example.studyplanner;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.List;

public class ToDo {
    private Integer id;
    private String title;
    private String description;
    private Integer priority;

    public ToDo(Integer id, String title, String description, Integer priority) {
        this.id = id;
        this.title = title;
        this.description = description;
        this.priority = priority;
    }
}
```

```

    public Integer getId() {
        return id;
    }

    public String getTitle() {
        return title;
    }

    public Integer getPriority() {
        return priority;
    }

    @Override
    public String toString() {
        return "ToDo {" +
            "id=" + id +
            ", title='" + title + '\'' +
            ", description='" + description + '\'' +
            ", priority=" + priority +
            '}';
    }

    public String getDetailedInfo(List<LocalDateTime> executionTimes) {
        StringBuilder str = new StringBuilder();
        str.append(toString()).append("\n");
        if (executionTimes == null || executionTimes.isEmpty()) {
            str.append("No tracks found\n");
        } else {
            for (LocalDateTime dateTime : executionTimes) {
                str.append(formatDate(dateTime)).append("\n");
            }
        }
        return str.toString();
    }

    private String formatDate(LocalDateTime dateTime) {
        String pattern = "yyyy-MM-dd HH:mm:ss";
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern(pattern);
        return formatter.format(dateTime);
    }
}

```

//REFATORAÇÃO DA IA TodoTracker

```

package org.example.studyplanner;

import java.time.LocalDateTime;
import java.util.*;

public class TodoTracker {
    private List<ToDo> toDos = new ArrayList<>();
    private final List<ToDo> completedToDos = new ArrayList<>();
    private Map<Integer, List<LocalDateTime>> tracker;
    private Integer nextId;
    private static TodoTracker instance;

    private TodoTracker() {
        this.tracker = new HashMap<>();
        this.toDos = new ArrayList<>();
        this.nextId = 1;
    }

    public void markCompleted(ToDo toDo) {
        if (toDo != null && !completedToDos.contains(toDo)) {

```

```

        completedTodos.add(todo);
        System.out.println("ToDo marked as completed: " + todo.getTitle());
    }
}

public List<ToDo> getCompletedTodos() {
    return completedTodos;
}

public static TodoTracker getInstance() {
    if (instance == null) {
        instance = new TodoTracker();
    }
    return instance;
}

@Override
public String toString() {
    StringBuilder str = new StringBuilder();
    for (ToDo todo : todos) {
        List<LocalDateTime> executionTimes = tracker.get(todo.getId());
        str.append(todo.getDetailedInfo(executionTimes));
    }
    String response = str.toString();
    return response.isEmpty() ? "No Todos found" : response;
}

public void addToDoExecutionTime(Integer id) {
    List<LocalDateTime> et = tracker.computeIfAbsent(id, k -> new ArrayList<>());
    LocalDateTime now = LocalDateTime.now();
    et.add(now);
}

public List<ToDo> getTodos() {
    return todos;
}

public ToDo getToDoById(Integer id) {
    for (ToDo todo : todos) {
        if (todo.getId() == id) {
            return todo;
        }
    }
    return null;
}

public Integer addToDo(String title, String description, Integer priority) {
    ToDo toAdd = new ToDo(nextId, title, description, priority);
    nextId++;
    this.todos.add(toAdd);
    return toAdd.getId();
}

public void removeToDo(Integer id) {
    todos.removeIf(todo -> todo.getId() == id);
}

public List<ToDo> sortTodosByPriority() {
    List<ToDo> sortedTodos = new ArrayList<>(todos);
    sortedTodos.sort(Comparator.comparingInt(ToDo::getPriority));
    return sortedTodos;
}

public List<String> searchInTodos(String search) {

```

```

        List<String> todos = new ArrayList<>();
        for (ToDo toDo : todos) {
            if (toDo.getTitle().toLowerCase().contains(search.toLowerCase()) || toDo.getDescription().toLowerCase().contains(search.toLowerCase())) {
                todos.add(toDo.toString());
            }
        }
        return todos;
    }
}

```

2. Pedi pra incorporar no código de ToDo pois ainda não tinha enviado o código a LLM

Alterações Implementadas

1. Movido o Método `appendToDoDetails` para `ToDo`:

- Foi renomeado para `getDetailedInfo` e adaptado para exibir informações do `ToDo` e seus horários de execução.

2. Movido o Método `appendToDoExecutionTimes` para `ToDo`:

- Foi incorporado ao método `getDetailedInfo`, eliminando redundâncias.

3. Refatorado o Método `toString` em `ToDoTracker`:

- Agora delega a exibição detalhada para `ToDo`.

Essa abordagem aumenta a coesão da classe `ToDo` e reduz a responsabilidade excessiva da classe `ToDoTracker`.

Incorpore as modificações de `ToDo` a seu código:

```
package org.example.studyplanner;
```

```
import java.text.MessageFormat;
import java.util.Objects;
```

```
public class ToDo implements PlannerMaterial {
    private Integer id;
    private String title;
    private String description;
    private int priority;
```

```
    public ToDo(Integer id, String title, String description, int priority) {
        this.id = id;
        this.title = title;
        this.description = description;
        this.priority = priority;
    }
}
```

Envie uma mensagem para o ChatGPT

Com as modificações já incluídas na classe `ToDo` que você forneceu, podemos integrar os métodos anteriormente transferidos para a classe `ToDo` com o código atualizado. O objetivo é que a lógica relacionada ao `ToDo` (como exibir detalhes e execução) permaneça coesa na própria classe.

Aqui está a versão atualizada da classe `ToDo` com as mudanças incorporadas:

Classe `ToDo` Atualizada

java

 Copiar código

```
package org.example.studyplanner;
```

```
import java.text.MessageFormat;
```

```
import java.time.LocalDateTime;
```

```
import java.time.format.DateTimeFormatter;
```

```

package org.example.studyplanner;

import java.text.MessageFormat;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.List;

public class ToDo implements PlannerMaterial {
    private Integer id;
    private String title;
    private String description;
    private int priority;

    public ToDo(Integer id, String title, String description, int priority) {
        this.id = id;
        this.title = title;
        this.description = description;
        this.priority = priority;
    }

    @Override
    public String toString() {
        return MessageFormat.format("[(Priority:{3}) ToDo {0}: {1}, {2}]", id, title, description, pr
iority);
    }

    /**
     * Adds domain logic to determine if the ToDo item is high priority.
     */
    public boolean isHighPriority() {
        return priority > 7;
    }

    /**
     * Updates the description with a timestamp (encapsulates description logic).
     */
    public void updateDescription(String newDescription) {
        this.description = newDescription + " (Updated at: " + System.currentTimeMillis() + ")";
    }

    /**
     * Generates detailed information about the ToDo, including execution times.
     */
    public String getDetailedInfo(List<LocalDateTime> executionTimes) {
        StringBuilder str = new StringBuilder();
        str.append(toString()).append("\n");
        if (executionTimes == null || executionTimes.isEmpty()) {
            str.append("No tracks found\n");
        } else {
            for (LocalDateTime dateTime : executionTimes) {
                str.append(formatDate(dateTime)).append("\n");
            }
        }
        return str.toString();
    }

    private String formatDate(LocalDateTime dateTime) {
        String pattern = "yyyy-MM-dd HH:mm:ss";
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern(pattern);
        return formatter.format(dateTime);
    }

    // Getters and setters
    public int getId() {

```

```

        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public int getPriority() {
        return priority;
    }

    public void setPriority(int priority) {
        this.priority = priority;
    }

    /**
     * Moves responsibility for tracking ToDo items to ToDoTracker.
     */
    public void markAsCompleted(ToDoTracker tracker) {
        tracker.markCompleted(this);
    }

    /**
     * Encapsulates logic to assign this ToDo to a habit using HabitTracker.
     */
    public void assignToHabit(HabitTracker tracker, Habit habit) {
        tracker.assignToHabit(this, habit);
    }
}

```

O que foi incorporado?

1. Método `getDetailedInfo`:

- Permite que a classe `ToDo` gere informações detalhadas sobre o item, incluindo seus horários de execução.
- Usa `formatDate` para padronizar a formatação de datas.

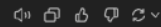
2. Método `formatDate`:

- Adicionado para formatar datas no padrão `yyyy-MM-dd HH:mm:ss`.

3. Lógica Centralizada:

- A responsabilidade de exibir informações detalhadas foi centralizada na classe `ToDo`.
- Agora a classe `ToDoTracker` pode delegar essa tarefa.

Essa abordagem mantém a responsabilidade de exibição e manipulação das informações relacionadas ao `ToDo` dentro de sua própria classe, seguindo os princípios de coesão e responsabilidade única.

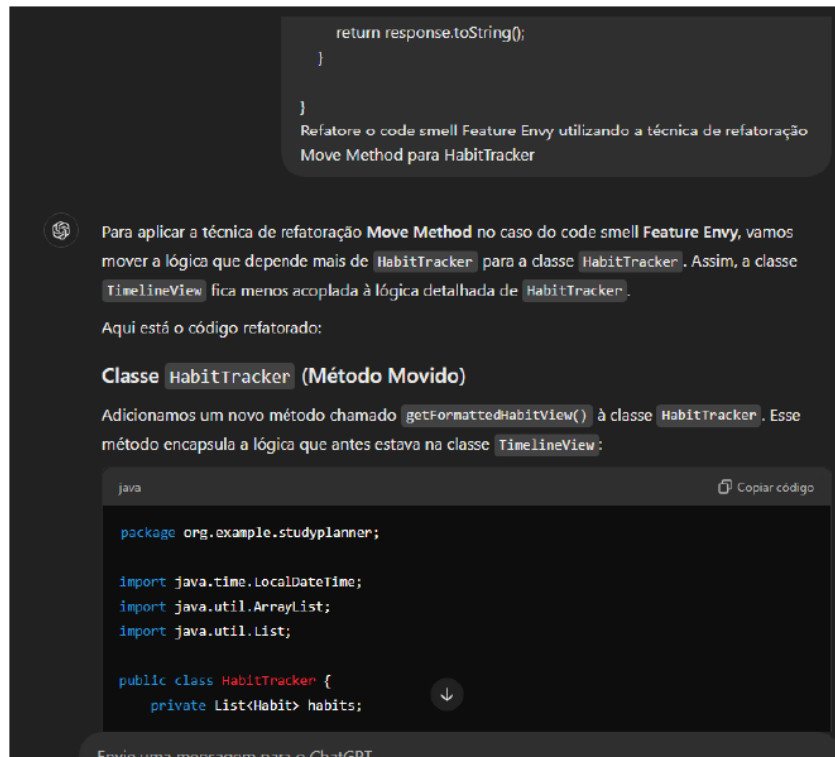


3. Código livre do code smell e funcionando

Feature Envy TimelineView

TimelineView

1. Foi enviado para a IA o código TimelineView com o seguinte comando:



```
//REFATORAÇÃO DA IA  
  
package org.example.studyplanner;  
  
import java.time.LocalDateTime;  
import java.util.ArrayList;  
import java.util.List;  
  
public class HabitTracker {  
    private List<Habit> habits;  
  
    public List<Habit> getHabits() {  
        return habits;  
    }  
  
    public List<LocalDateTime> getHabitRecords(int habitId) {  
        // Retorna os registros do hábito com o ID fornecido  
        return new ArrayList<>();  
    }  
  
    public String formatHabitDate(LocalDateTime date) {  
        // Formata a data conforme o formato esperado  
        return date.toString();  
    }  
  
    public String getFormattedHabitView() {  
        StringBuilder response = new StringBuilder();  
        for (Habit habit : habits) {  
            response.append("[ Habit: ")
```

```

        .append(habit.getName())
        .append(". Records: ");
List<LocalDateTime> records = getHabitRecords(habit.getId());
for (LocalDateTime record : records) {
    response.append(formatHabitDate(record)).append(", ");
}
response.append("]");
}
return response.toString();
}
}
}

```

Classe `TimelineView` (Após a Refatoração)

Agora a classe `TimelineView` delega a tarefa para o método `getFormattedHabitView()` da classe `HabitTracker`:

```

java Copiar código

package org.example.studyplanner;

public class TimelineView {

    public String habitDateViewAll(HabitTracker ht) {
        // Apenas delega a responsabilidade ao método de HabitTracker
        return ht.getFormattedHabitView();
    }
}

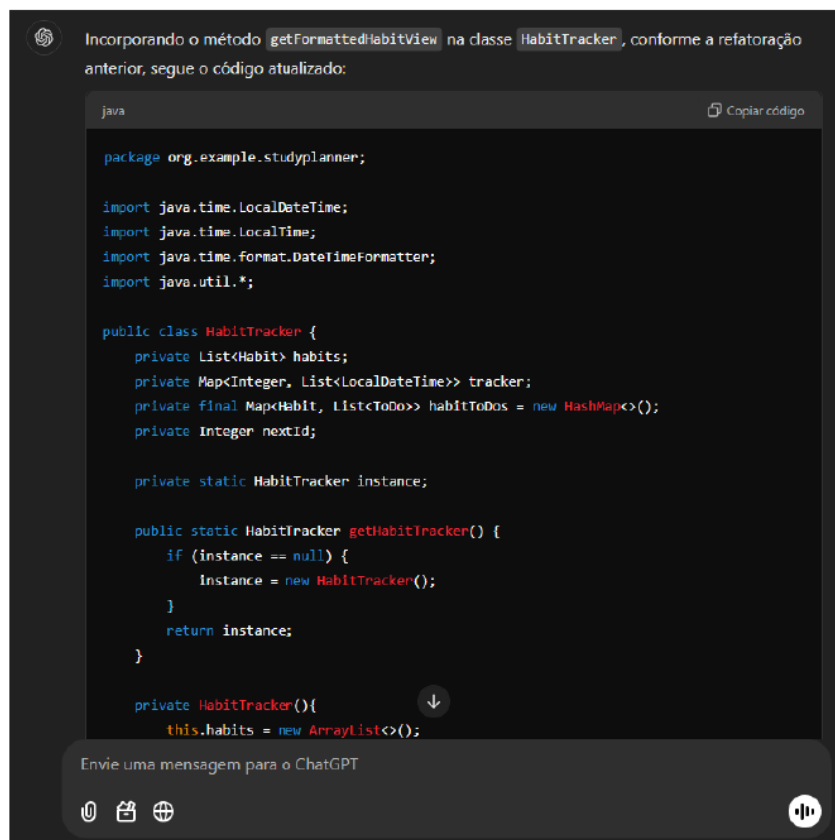
```

Benefícios da Refatoração

1. **Encapsulamento:** A lógica relacionada à formatação de hábitos e seus registros foi movida para a classe `HabitTracker`, onde os dados residem, mantendo as responsabilidades mais claras.
2. **Reutilização:** O método `getFormattedHabitView()` pode ser reutilizado em outras partes do sistema sem precisar passar pela classe `TimelineView`.
3. **Facilidade de Testes:** Agora, é mais fácil testar a lógica de formatação de hábitos isoladamente na classe `HabitTracker`.

Essa refatoração segue o princípio de coesão, mantendo os métodos próximos aos dados que eles manipulam.

2. Pedi pra incorporar no código de `HabitTracker` pois não estava funcionando



```
package org.example.studyplanner;
```

```

import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
import java.util.*;

public class HabitTracker {
    private List<Habit> habits;
    private Map<Integer, List<LocalDateTime>> tracker;
    private final Map<Habit, List<ToDo>> habitToDos = new HashMap<>();
    private Integer nextId;

    private static HabitTracker instance;

    public static HabitTracker getHabitTracker() {
        if (instance == null) {
            instance = new HabitTracker();
        }
        return instance;
    }

    private HabitTracker(){
        this.habits = new ArrayList<>();
        this.tracker = new HashMap<>();
        this.nextId = 1;
    }

    @Override
    public String toString() {
        StringBuilder response = new StringBuilder();
        for (Habit habit : habits) {
            response.append(habit.toString()).append(", ");
        }
        return "Habits: " + response.toString();
    }

    public Habit getHabitById(Integer id){
        return this.habits.stream()
            .filter(habit -> Objects.equals(habit.getId(), id))
            .findFirst().orElse(null);
    }

    public void assignToHabit(ToDo toDo, Habit habit) {
        habitToDos.computeIfAbsent(habit, k -> new ArrayList<>()).add(toDo);
        System.out.println("ToDo '" + toDo.getTitle() + "' assigned to Habit: " + habit.getName());
    }

    public List<ToDo> getToDosForHabit(Habit habit) {
        return habitToDos.getOrDefault(habit, new ArrayList<>());
    }

    public List<Habit> getHabits() {
        return this.habits;
    }

    public String formatHabitDate(LocalDateTime date){
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");

        return date.format(formatter);
    }

    public List<Integer> getTrackerKeys(){
        return this.tracker.keySet().stream().toList();
    }
}

```

```

    public int addHabit(String name, String motivation, Integer dailyMinutesDedication, Integer daily
HoursDedication, Integer year, Integer month, Integer day, Integer hour, Integer minute, Integer seco
nds, Boolean isConcluded) {
        LocalDateTime lt = LocalDateTime.of(dailyHoursDedication, dailyMinutesDedication);
        LocalDateTime startDate = LocalDateTime.of(year, month, day, hour, minute, seconds);
        Habit habit = new Habit(name, motivation, lt, this.nextId, startDate, isConcluded);
        this.habits.add(habit);
        int response = nextId;
        this.tracker.put(nextId, new ArrayList<>());
        this.nextId++;
        return response;
    }

    public int handleAddHabitAdapter(List<String> stringProperties, List<Integer> intProperties, bool
ean isConcluded){
        return addHabit(stringProperties.get(0), stringProperties.get(1), intProperties.get(0), intPr
operties.get(1), intProperties.get(2), intProperties.get(3), intProperties.get(4), intProperties.get
(5), intProperties.get(6), intProperties.get(7), isConcluded);
    }

    public int addHabit(String name, String motivation) {

        Habit habit = new Habit(name, motivation, this.nextId);
        this.habits.add(habit);
        int response = nextId;
        this.tracker.put(nextId, new ArrayList<>());
        this.nextId++;
        return response;
    }

    public void addHabitRecord(Integer id){
        tracker.get(id).add(LocalDateTime.now());
    }

    public void toggleConcludeHabit(Integer id) {
        for (Habit habit : this.habits) {
            if (habit.getId().equals(id)) {
                habit.setIsConcluded(!habit.getIsConcluded());
            }
        }
    }

    public void removeHabit(Integer id) {
        this.habits.removeIf(habit -> habit.getId().equals(id));
        this.tracker.remove(id);
    }

    public List<LocalDateTime> getHabitRecords(Integer id) {
        return this.tracker.get(id);
    }

    public List<String> searchInHabits(String search){
        List<String> habits = new ArrayList<>();
        for (Habit habit : this.habits) {
            if (habit.getName().toLowerCase().contains(search.toLowerCase()) || habit.getMotivation
().toLowerCase().contains(search.toLowerCase())) {
                habits.add(habit.toString());
            }
        }
        return habits;
    }

    // Método Movido: getFormattedHabitView
    public String getFormattedHabitView() {

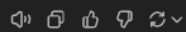
```

```

        StringBuilder response = new StringBuilder();
        for (Habit habit : habits) {
            response.append("[ Habit: ")
                .append(habit.getName())
                .append(". Records: ");
            List<LocalDateTime> records = getHabitRecords(habit.getId());
            for (LocalDateTime record : records) {
                response.append(formatHabitDate(record)).append(", ");
            }
            response.append("]");
        }
        return response.toString();
    }
}

```

Agora, a lógica de exibição de hábitos e registros está incorporada diretamente na classe `HabitTracker`, seguindo o princípio da responsabilidade única e melhorando o encapsulamento.



3. Código livre do code smell e funcionando